API REST de productos (Tienda)

Desarrollo de Software II — Semana 1

Asignatura	Desarrollo de Software II
Docente	MSc. Viviana Flores
Actividad	Actividad 1: API REST de productos (Tienda)
Modalidad	Individual
Valor	10 puntos
Estudiante	Raúl Fernando Cajiao Garcés
Semestre	7mo Semestre

1. Objetivo

Construir una API REST profesional para la gestión del catálogo de productos de una tienda utilizando Node.js, Express y TypeScript, aplicando principios de Clean Architecture y Clean Code, validando rigurosamente los datos de entrada, escribiendo pruebas automatizadas con Jest y Supertest, configurando la colección de pruebas en Postman (Collection v2.1) y ejecutando el flujo completo de trabajo bajo GitHub Flow.

2. Qué se construyó

Se diseñó e implementó la arquitectura modular en `src/modules/products`, garantizando el desacoplamiento entre el núcleo del negocio y los adaptadores de infraestructura:

Capa	Archivo	Responsabilidad
Dominio	`product.entity.ts`	Modelo de producto (`id`, `name`, `price`, `stock`, `category`) y reglas de validación en español.
Puerto	`product.repository.ts`	Contrato e interfaz con `findAll`, `findById`, `save` y `reset`.
Adaptador salida	`in-memory-product.repository.ts`	Repositorio en memoria con 2 productos iniciales e inmutabilidad con copias seguras.

Aplicación	`get-products.usecase.ts`
	`create-product.usecase.ts`
	`get-product-by-id.usecase.ts`
		Casos de uso desacoplados para listar el catálogo, crear un nuevo producto y consultar por ID.

Adaptador entrada	`product.controller.ts` `product.routes.ts`	Controladores y enrutadores Express con gestión de códigos HTTP (200, 201, 400, 404).

Ensamblado	`app.ts`
	`server.ts`
	`server.js`
		Configuración de Express, inyección de dependencias y exportación para pruebas sin levantar puertos.

3. Reglas de negocio

La entidad de dominio `Product` valida estrictamente los campos obligatorios antes de permitir su almacenamiento:

Regla (en el dominio)	Mensaje de error para el usuario
`name` y `price` son obligatorios	*"El nombre y el precio son obligatorios"*
`price` no es numérico o es ≤ 0	*"El precio debe ser un número mayor que 0"*
Identificador no entero en `:id`	*"El identificador debe ser un número entero"*
Producto inexistente en `:id`	*"Producto no encontrado"*

4. Comportamiento de la API

La API maneja todas las respuestas mediante códigos de estado HTTP canónicos:

Operación	Ruta	Éxito	Error
Listar	`GET /products`	200 con el arreglo JSON	500: Error interno
Crear	`POST /products`	201 con el producto creado y su ID	400 con mensaje específico de validación
Ver uno (Reto)	`GET /products/:id`	200 con el producto	400 (ID inválido) o 404: Producto no encontrado

Datos de prueba iniciales cargados en memoria:

- Producto 1: `id: 1, name: 'Camiseta básica', price: 15.5, stock: 40, category: 'ropa'`
- Producto 2: `id: 2, name: 'Taza de cerámica', price: 6.9, stock: 120, category: 'hogar'`

5. Pruebas automatizadas (10 pruebas en verde)

Se implementaron 10 pruebas automatizadas con Jest y Supertest en el archivo de integración `src/modules/products/products.test.ts`, superando ampliamente el mínimo de 4 solicitado:

1. `GET /products`: responde 200 y devuelve un arreglo con los 2 productos iniciales.
2. `POST /products`: responde 201 y devuelve el producto creado con su ID para datos válidos.
3. `POST /products`: responde 400 si falta el campo `name`.
4. `POST /products`: responde 400 si falta el campo `price`.
5. `POST /products`: responde 400 si el precio es igual a 0.
6. `POST /products`: responde 400 si el precio es negativo.
7. `POST /products`: responde 400 si el precio es texto.

8. `GET /products/:id`: responde 200 y el objeto correspondiente si el ID existe (Reto opcional).
9. `GET /products/:id`: responde 404 con mensaje en español si el ID no existe (Reto opcional).
10. `GET /products/:id`: responde 400 si el parámetro ID no es numérico entero.

6. Requisitos de GitHub Flow

- **Rama principal protegida**: Inicialización en `main` con el commit inicial de configuración y dependencias.
- **Rama de trabajo**: Creación de la rama `feature/products-endpoints`.
- **Commits atómicos**: 8 commits semánticos siguiendo *Conventional Commits* (`chore:`, `feat:`, `test:`, `docs:`).
- **Pull Request**: Creación del PR #1 con descripción técnica (*Qué se hizo, Por qué, Cómo probarlo*).
- **Merge y Limpieza**: Fusión completa del PR hacia `main` y eliminación de la rama de características tanto en remoto como en local.

7. Colección de Postman y Bruno

Se exportó la colección en formato estándar **Postman Collection v2.1** denominada `Store API`, conteniendo la variable de entorno `baseUrl = http://localhost:3000` y scripts de aserción automatizados en cada una de las peticiones requeridas:

- **List products**: `GET {{baseUrl}}/products` → Test: código 200 y verificación de arreglo.
- **Create product**: `POST {{baseUrl}}/products` → Test: código 201 y validación de propiedades.
- **Create invalid product**: `POST {{baseUrl}}/products` → Test: código 400 y mensaje en español.
- **Get product by ID**: `GET {{baseUrl}}/products/1` → Test: código 200 e ID coincidente.

8. Evidencias y Enlaces de Entrega

Entregable Requerido	Detalle / Enlace
1. Repositorio de GitHub	https://github.com/Raul-Fernando-Cajiao-Garces/store-products-api
2. Pull Request (Merged)	https://github.com/Raul-Fernando-Cajiao-Garces/store-products-api/pull/1
3. Colección Postman v2.1	Archivo adjunto: `Store API.postman_collection.json`
4. Pruebas (`npm test`)	100 % pruebas aprobadas en verde (10 tests pasados con Jest y Supertest)
5. Compilación (`npm run build`)	Compilación TypeScript limpia sin errores en `dist/`

Historial Gráfico de Git (`git log --oneline --graph --all`):

```
*   5912e05 (HEAD -> main, origin/main) Merge pull request #1 from Raul-Fernando
    -Cajiao-Garces/feature/products-endpoints
|\
| * 921375b (origin/feature/products-endpoints) docs: add postman collection v2
    .1 and comprehensive project documentation
| * 88358e5 test(products): add automated integration test suite with jest and
    supertest
| * 996ec33 feat(app): configure express application, routes and server entry
    point
| * 896095d feat(http): implement express controllers and routes with validation
     handling
| * 92f886e feat(application): implement get, create and find by id use cases
| * 358461c feat(adapters): implement in-memory product repository with safe
    copies
| * 31ee280 feat(ports): define product repository port interface
| * ce9b035 feat(domain): define product entity and validation rules
|/
* 771b40b chore: initial project configuration and dependencies
```

Evidencia de Ejecución de Pruebas (`npm test`):

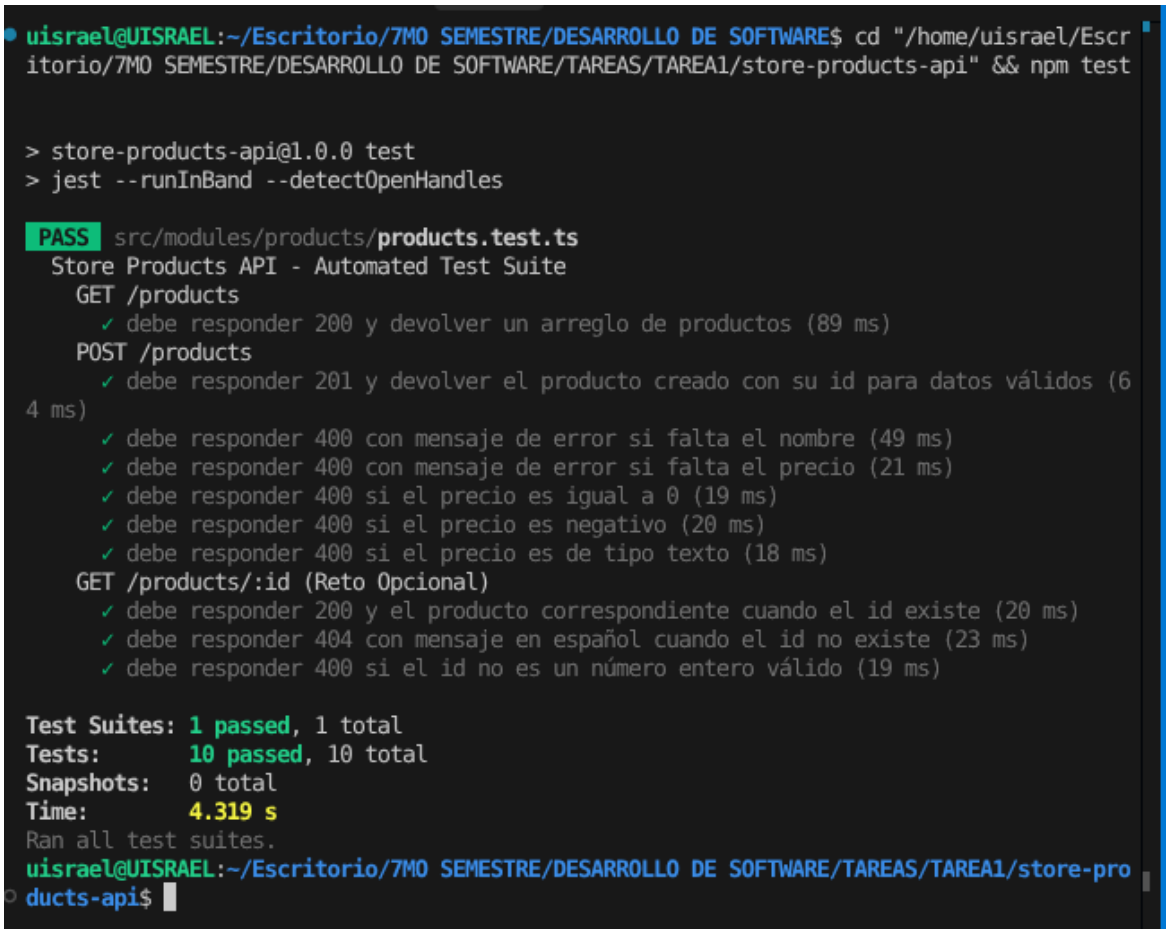


Figura 1: Captura de pantalla de la terminal con las 10 pruebas automatizadas pasando 100 % en verde (`npm test`).